



**Escuela Politécnica Nacional
Facultad de Ingeniería de Sistemas
Construcción y evolución de software**

PLANIFICACIÓN DE LA CONSTRUCCIÓN Y EVOLUCIÓN DE LA COCINA DE GREGORY

Integrantes:

Fernando Huilca
Mateo Simbaña

Fecha de entrega:

31 de enero de 2026

Índice

1.	Introducción	3
2.	Arquitectura del proyecto	3
2.1	Descripción general de la arquitectura.....	3
2.2	Componentes del sistema.....	3
2.3	Estrategia de integración.....	4
3.	Estrategia de Pipelines (CI/CD).....	4
3.1	Pipeline de integración continua.....	4
3.2	Pipeline de entrega continua	5
4.	Estrategia de flujos de desarrollo.....	6
5.	Gestión de historias de usuario	6
6.	Estrategia de revisiones y aprobaciones	7
7.	Herramientas y conexiones	8
8.	Conclusiones.....	9

1. Introducción

La aplicación web *La cocina de Gregory* será un sistema orientado a la gestión de recetas de cocina, cuyo propósito será permitir a los usuarios registrar, consultar, modificar y eliminar recetas de manera organizada. El sistema se plantea como una solución web que facilite la administración de información gastronómica mediante una interfaz intuitiva y funcionalidades claramente definidas.

El objetivo de este documento es definir la estrategia de construcción y evolución del software, describiendo la arquitectura propuesta, los flujos de desarrollo, la integración continua, la gestión de versiones y las buenas prácticas que se aplicarán durante el ciclo de vida del proyecto para garantizar su calidad y mantenibilidad.

2. Arquitectura del proyecto

2.1 Descripción general de la arquitectura

La aplicación *La cocina de Gregory* se construirá siguiendo una arquitectura web basada en el modelo MVC (Modelo-Vista-Controlador). Este enfoque permitirá separar la lógica de presentación, la lógica de negocio y el acceso a datos, facilitando el mantenimiento del sistema y su evolución futura.

2.2 Componentes del sistema

El sistema estará compuesto por los siguientes componentes principales:

- **Frontend (Vista):** Será desarrollado utilizando páginas JSP, HTML y CSS, permitiendo la interacción del usuario con el sistema mediante formularios y vistas dinámicas.
- **Backend (Controlador y lógica de negocio):** Será implementado en Java, utilizando controladores que gestionarán las peticiones del usuario, validarán la información ingresada y coordinarán las operaciones del sistema.
- **Base de datos:** Se utilizará MySQL como sistema de gestión de base de datos para almacenar de forma persistente la información relacionada con usuarios, recetas e ingredientes.

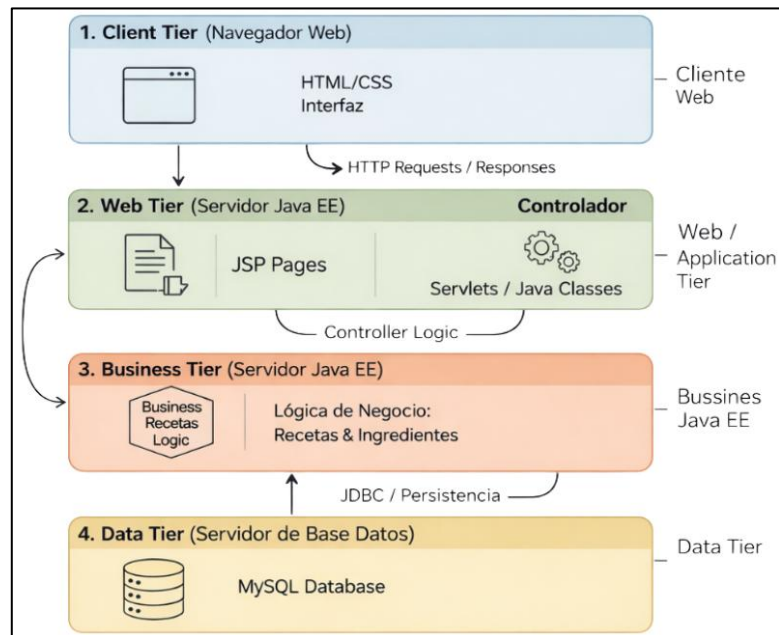


Ilustración 1. Arquitectura web MVC de la aplicación.

2.3 Estrategia de integración

El frontend se comunicará con el backend a través de solicitudes HTTP manejadas por los controladores del sistema. El backend procesará la lógica de negocio y accederá a la base de datos mediante entidades y gestores de persistencia. Los resultados obtenidos serán enviados nuevamente al frontend para su visualización.

Esta estrategia permitirá desacoplar las capas del sistema, facilitando cambios futuros en la interfaz de usuario o en la lógica de negocio sin afectar al resto de componentes.

3. Estrategia de Pipelines (CI/CD)

3.1 Pipeline de integración continua

Se implementará un pipeline de integración continua con el objetivo de validar automáticamente los cambios realizados en el código fuente.

El pipeline de integración continua estará compuesto por los siguientes pasos:

- 1) Ejecución de validaciones de estilo de código (lint).
- 2) Ejecución de pruebas unitarias.
- 3) Compilación automática del proyecto (build).
- 4) Generación de artefactos para el entorno de pruebas.

Cada vez que se realice un push a la rama *develop*, el pipeline se ejecutará automáticamente para detectar errores de forma temprana.

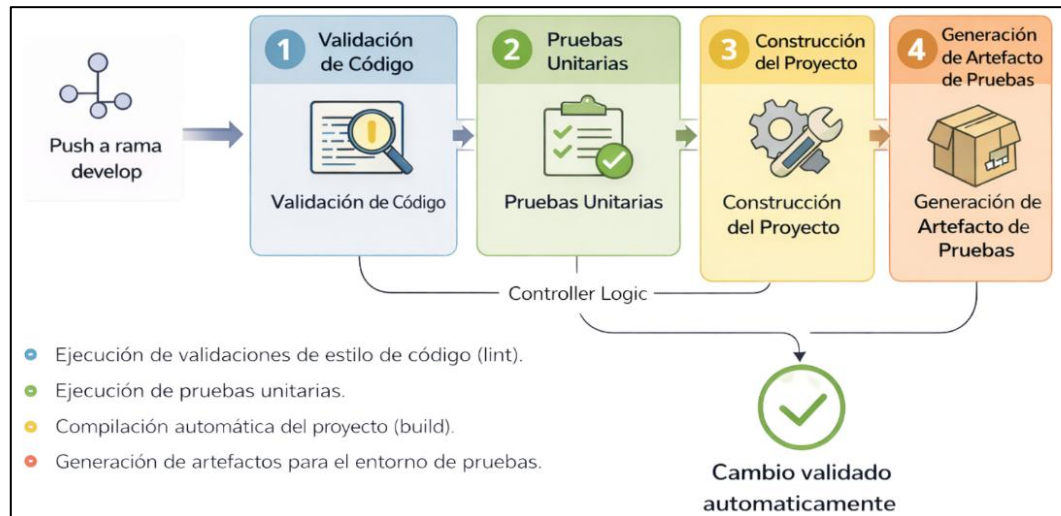


Ilustración 2. Estrategia de pipeline de integración continua.

3.2 Pipeline de entrega continua

La entrega continua se enfocará en el despliegue controlado del sistema hacia los distintos entornos.

El despliegue al entorno de producción se realizará únicamente después de la aprobación de un Pull Request, garantizando que el código haya sido revisado y validado previamente. Esta estrategia reducirá el riesgo de introducir fallos en el sistema productivo.

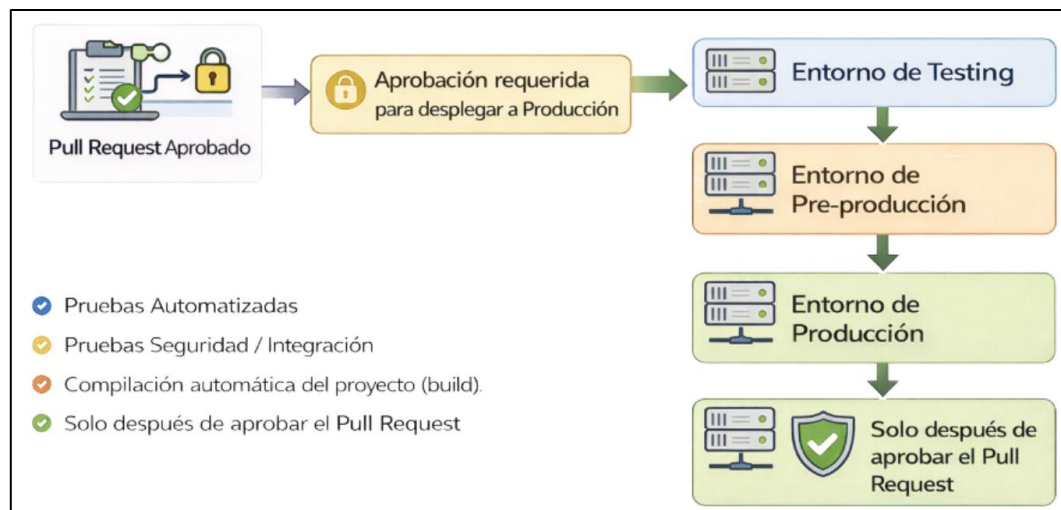


Ilustración 3. Estrategia de pipeline de entrega continua.

4. Estrategia de flujos de desarrollo

Para el desarrollo del proyecto se adoptará un modelo de ramas basado en buenas prácticas de control de versiones:

- **main:** Contendrá la versión estable del sistema.
- **develop:** Integrará las nuevas funcionalidades en desarrollo.
- **feature/*:** Se utilizarán para el desarrollo de historias de usuario específicas.
- **hotfix/*:** Se emplearán para correcciones urgentes.

Las nuevas funcionalidades se desarrollarán en ramas *feature*. Una vez completadas, se creará un Pull Request hacia la rama *develop*, donde el código será revisado antes de su integración.

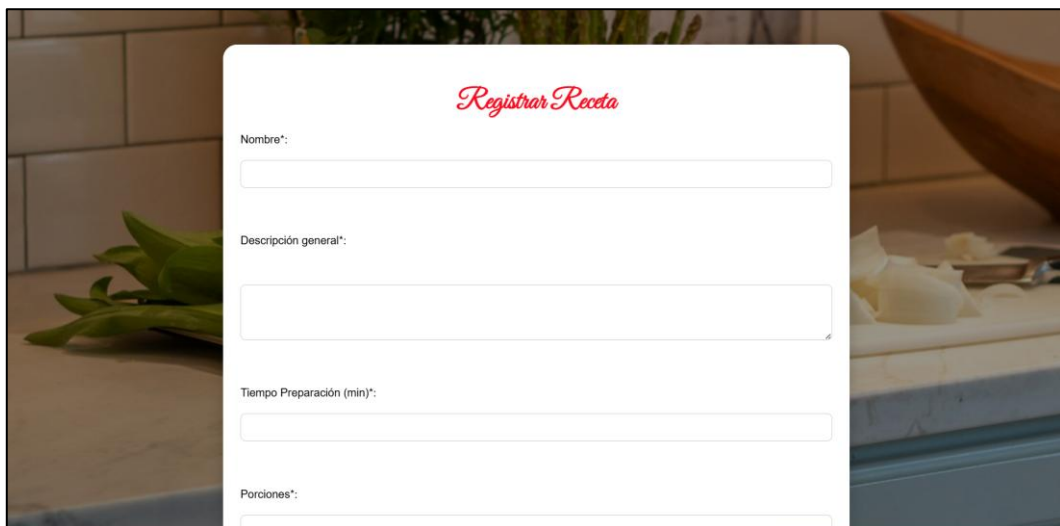
5. Gestión de historias de usuario

Las funcionalidades del sistema se definirán mediante historias de usuario, siguiendo el formato:

Como [rol], **quiero** [funcionalidad], **para** [beneficio].

Ejemplos de historias de usuario propuestas para el proyecto son:

- **Como** usuario, **quiero** registrar una receta, **para** almacenar mis preparaciones culinarias.



El prototipo muestra un formulario con el título "Registrar Receta" en rojo. El formulario está superpuesto sobre una imagen de fondo que muestra una cocina con una pila de verduras y un cuchillo. El formulario contiene los siguientes campos:

- Nombre*:
- Descripción general*:
- Tiempo Preparación (min)*:
- Porciones*:

Ilustración 4. Prototipo para el registro de una receta.

- **Como** usuario, **quiero** editar una receta, **para** mantener actualizada su información.

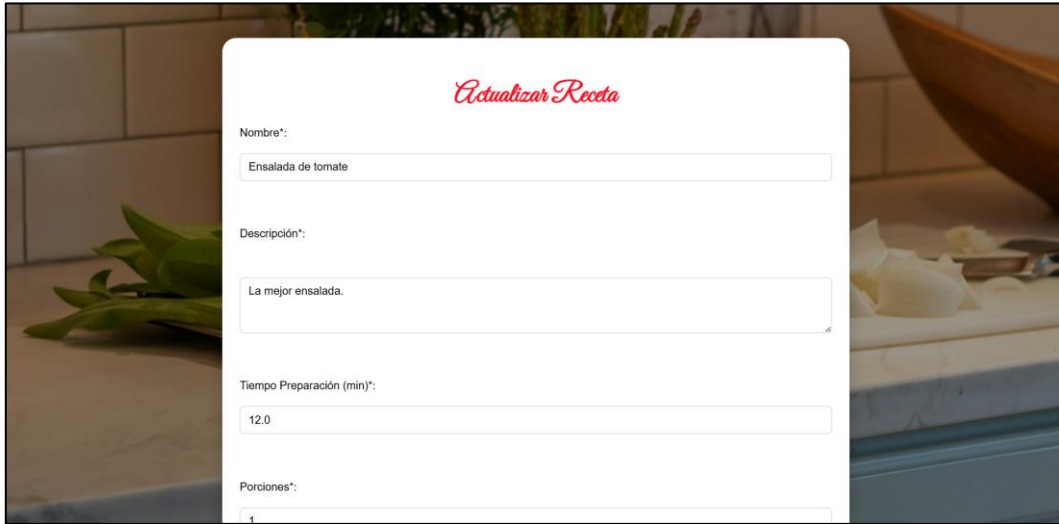


Ilustración 5. Prototipo para la actualización de una receta.

- **Como** administrador, **quiero** eliminar recetas, **para** garantizar la organización de la base de datos.



Ilustración 6. Prototipo para la gestión de recetas.

Las historias de usuario se gestionarán mediante herramientas como Azure DevOps, donde se asignarán prioridades, responsables y se organizarán en sprints de dos semanas.

6. Estrategia de revisiones y aprobaciones

Todo cambio en el código se realizará mediante Pull Requests. Cada Pull Request deberá incluir una descripción clara de los cambios propuestos y será revisado por al menos un integrante del equipo antes de su aprobación.

Se aplicará un checklist de revisión que considerará:

- Cumplimiento de estándares de estilo.
- Ejecución exitosa de pruebas.
- Actualización de documentación cuando corresponda.

Esta estrategia permitirá asegurar la calidad del código y promover el trabajo colaborativo.

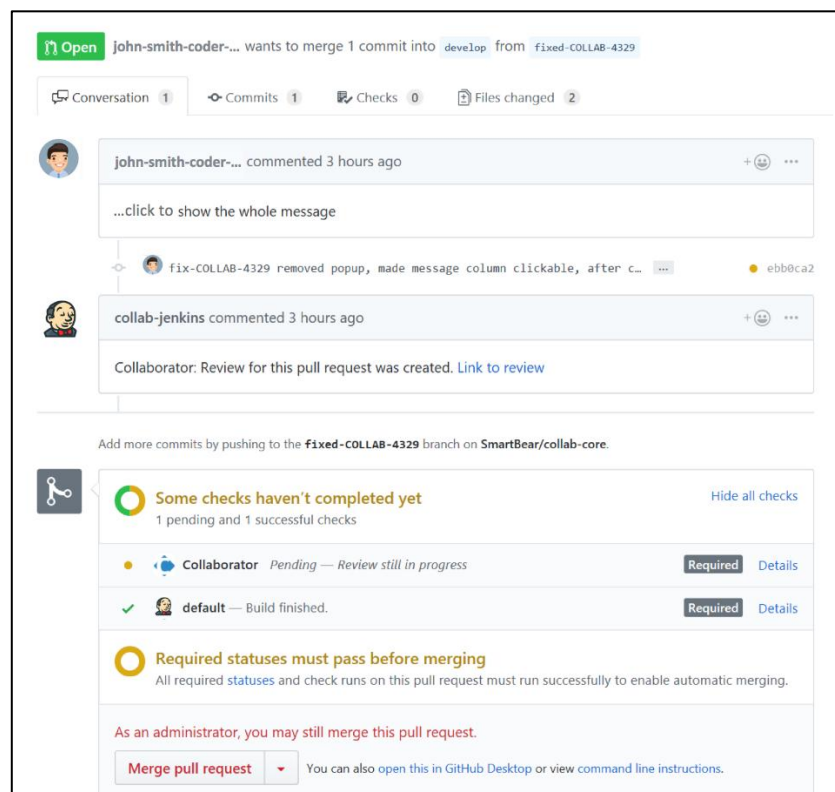


Ilustración 7. Flujo de revisión de código mediante Pull Request, con ejecución de checks y aprobación previa a la fusión.

7. Herramientas y conexiones

Para la construcción y evolución del proyecto se utilizarán las siguientes herramientas:

- **GitHub:** Para el control de versiones y gestión del repositorio.
- **GitHub Actions:** Para la automatización de pipelines de integración continua.
- **Azure DevOps:** Para la gestión de tareas e historias de usuario.
- **MySQL:** Como sistema de gestión de base de datos.
- **WhatsApp / Teams:** Para la comunicación entre los miembros del equipo.

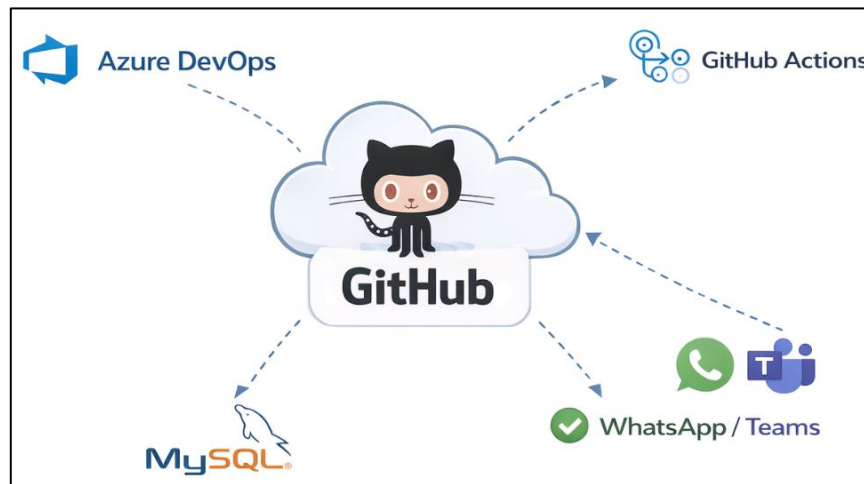


Ilustración 8. Herramientas tecnológicas para la construcción y evolución del proyecto.

Estas herramientas permitirán mantener trazabilidad entre tareas, cambios en el código y revisiones realizadas.

8. Conclusiones

- La definición anticipada de una arquitectura basada en el modelo MVC permitirá que la aplicación web *La cocina de Gregory* se construya de forma modular y ordenada, facilitando el mantenimiento del sistema y su adaptación a futuros cambios o nuevas funcionalidades sin afectar la estabilidad del software.
- La estrategia propuesta de control de versiones, flujos de desarrollo y uso de Pull Requests contribuirá a un proceso de construcción del software más controlado y colaborativo, reduciendo errores de integración y asegurando que cada cambio sea revisado antes de incorporarse a las ramas principales del proyecto.
- La implementación planificada de pipelines de integración y entrega continuas permitirá detectar errores de manera temprana, automatizar tareas repetitivas y mejorar la calidad del producto final, garantizando que la evolución del software se realice de forma progresiva, trazable y alineada con buenas prácticas de ingeniería de software.